

Assignment-6.3

Name: D. Naveen

2403A53L01

Batch:46

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Lab Objectives

- To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements.
- To analyze how AI tools suggest logic for object-oriented programming and control structures.
- To evaluate the correctness, readability, and completeness of AI-generated Python code.

Lab Outcomes (LOs)

After completing this lab, students will be able to:

- Use AI tools to generate and complete Python class definitions and methods.
- Understand and assess AI-suggested loop constructs for iterative tasks.
- Generate and evaluate conditional statements using AI-driven prompts.
- Critically analyze AI-assisted code for correctness, clarity, and efficiency.

Task Description #1: Classes (Student Class) Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

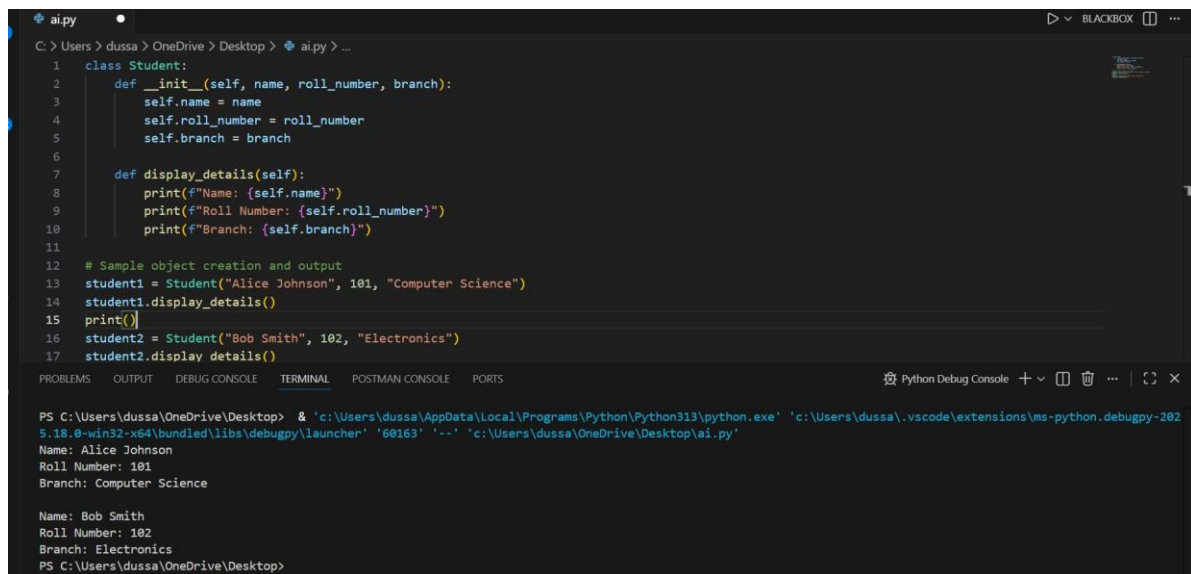
Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code

Prompt:

Generate a Python Student class with attributes name, roll number, and branch. Include a method to display student details. Also show sample object creation and output.

Code:



```
1 class Student:
2     def __init__(self, name, roll_number, branch):
3         self.name = name
4         self.roll_number = roll_number
5         self.branch = branch
6
7     def display_details(self):
8         print(f"Name: {self.name}")
9         print(f"Roll Number: {self.roll_number}")
10        print(f"Branch: {self.branch}")
11
12 # Sample object creation and output
13 student1 = Student("Alice Johnson", 101, "Computer Science")
14 student1.display_details()
15 print()
16 student2 = Student("Bob Smith", 102, "Electronics")
17 student2.display_details()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console

```
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '60163' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
Name: Alice Johnson
Roll Number: 101
Branch: Computer Science

Name: Bob Smith
Roll Number: 102
Branch: Electronics
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis:

The AI-generated class is correct and readable. It properly uses a constructor (`__init__`) to initialize attributes and a method to display details. Variable names are clear and follow Python conventions.

Task Description #2: Loops (Multiples of a Number) Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

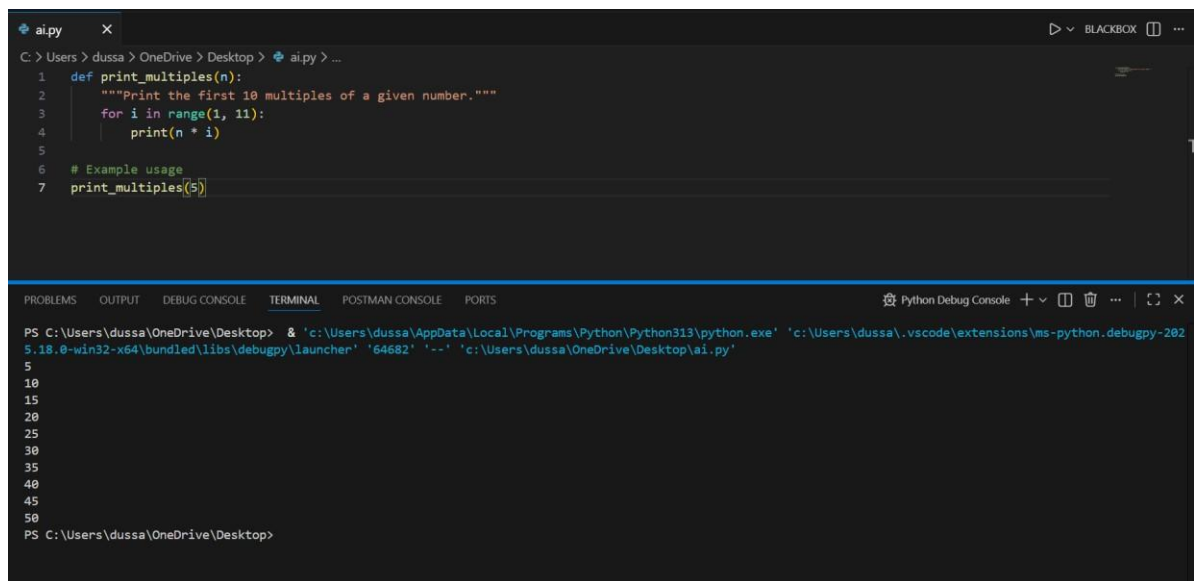
- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches

Prompt:

Write a Python function to print the first 10 multiples of a given number using a for loop.

Code:

Using for Loop:

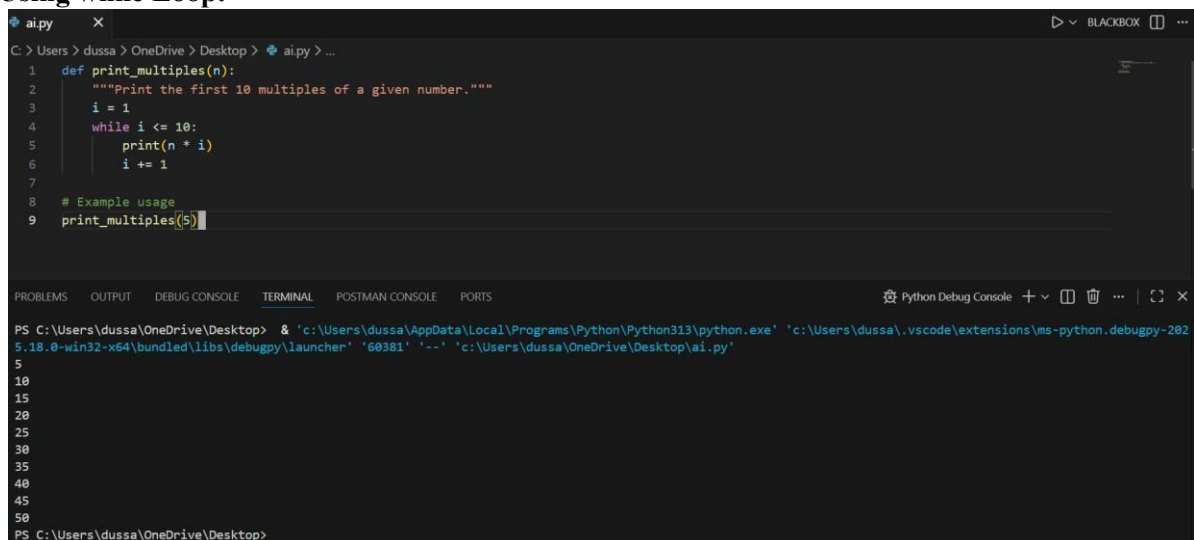


```
ai.py X
C:\Users\dussa> OneDrive\ Desktop > ai.py > ...
1 def print_multiples(n):
2     """Print the first 10 multiples of a given number."""
3     for i in range(1, 11):
4         print(n * i)
5
6 # Example usage
7 print_multiples(5)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64682' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
5
10
15
20
25
30
35
40
45
50
PS C:\Users\dussa\OneDrive\Desktop>
```

Code:

Using while Loop:



```
ai.py X
C:\Users\dussa> OneDrive\ Desktop > ai.py > ...
1 def print_multiples(n):
2     """Print the first 10 multiples of a given number."""
3     i = 1
4     while i <= 10:
5         print(n * i)
6         i += 1
7
8 # Example usage
9 print_multiples(5)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60381' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
5
10
15
20
25
30
35
40
45
50
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis

Both loops produce the same output. The for loop is more concise and readable, while the while loop provides more control over iteration.

Task Description #3: Conditional Statements (Age Classification) Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

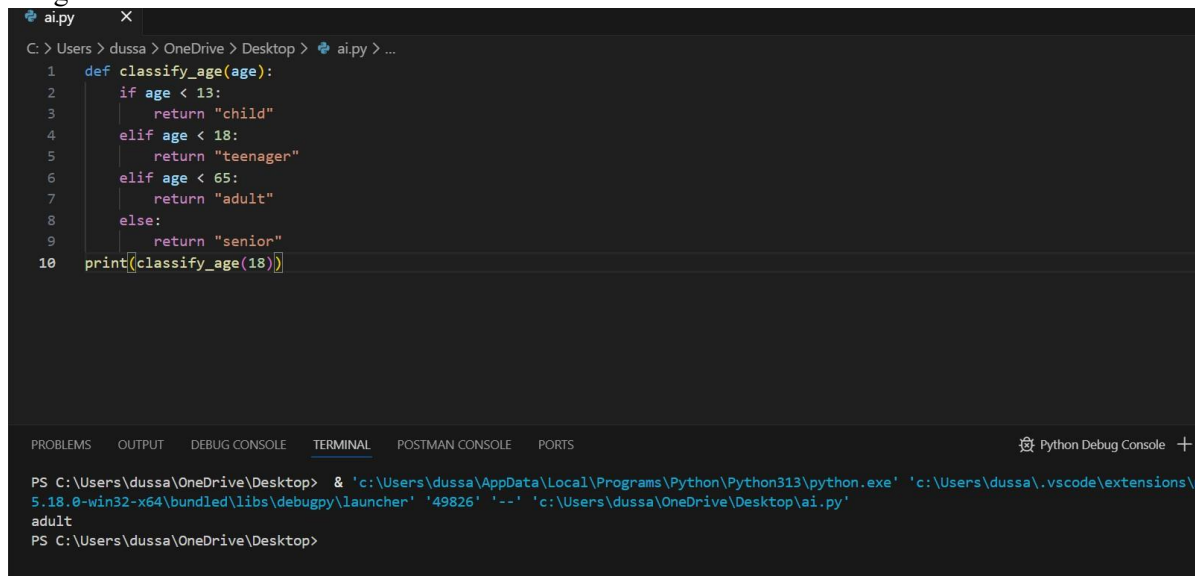
Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Prompt:

Generate a Python function using if-elif-else to classify age as child, teenager, adult, or senior.

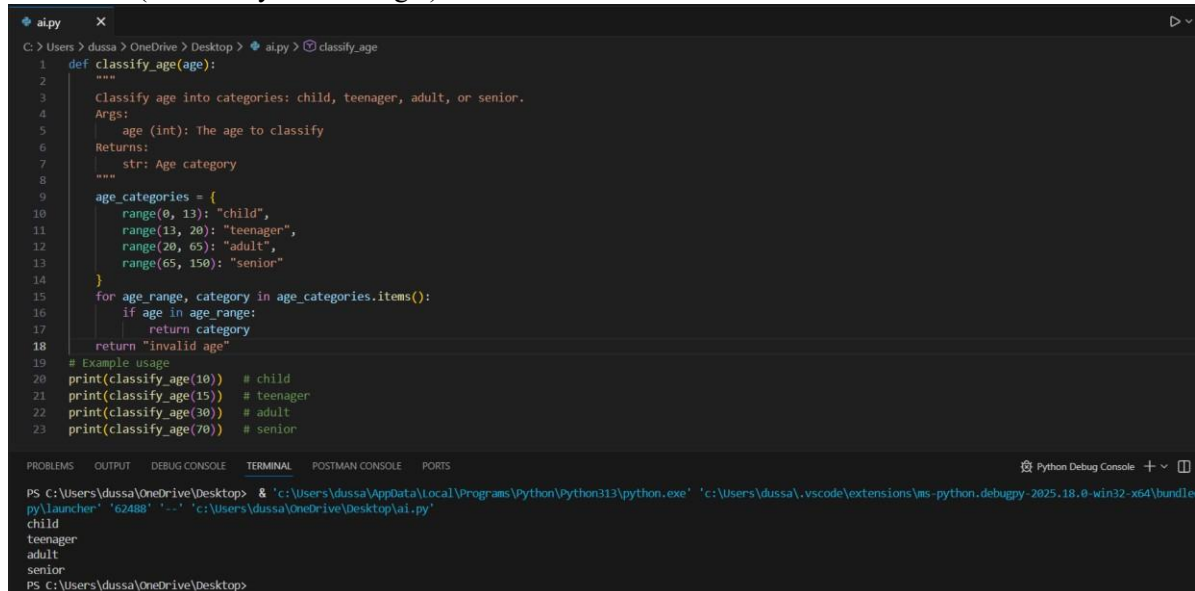
Using if-elif-else Code:



```
ai.py
C: > Users > dussa > OneDrive > Desktop > ai.py > ...
1 def classify_age(age):
2     if age < 13:
3         return "child"
4     elif age < 18:
5         return "teenager"
6     elif age < 65:
7         return "adult"
8     else:
9         return "senior"
10 print(classify_age(18))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console +
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
py\launcher' '49826' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
adult
PS C:\Users\dussa\OneDrive\Desktop>
```

Alternative (Dictionary-Based Logic)



```
ai.py
C: > Users > dussa > OneDrive > Desktop > ai.py > classify_age
1 def classify_age(age):
2     """
3     Classify age into categories: child, teenager, adult, or senior.
4     Args:
5     age (int): The age to classify
6     Returns:
7     str: Age category
8     """
9     age_categories = {
10         range(0, 13): "child",
11         range(13, 20): "teenager",
12         range(20, 65): "adult",
13         range(65, 150): "senior"
14     }
15     for age_range, category in age_categories.items():
16         if age in age_range:
17             return category
18     return "invalid age"
19 # Example usage
20 print(classify_age(10)) # child
21 print(classify_age(15)) # teenager
22 print(classify_age(30)) # adult
23 print(classify_age(70)) # senior

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS Python Debug Console +
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
py\launcher' '62488' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
child
teenager
adult
senior
PS C:\Users\dussa\OneDrive\Desktop>
```

Analysis

The if-elif-else structure is simpler and more readable. The dictionary-based approach is creative but slightly less intuitive for beginners.

Task Description #4: For and While Loops (Sum of First n Numbers) Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

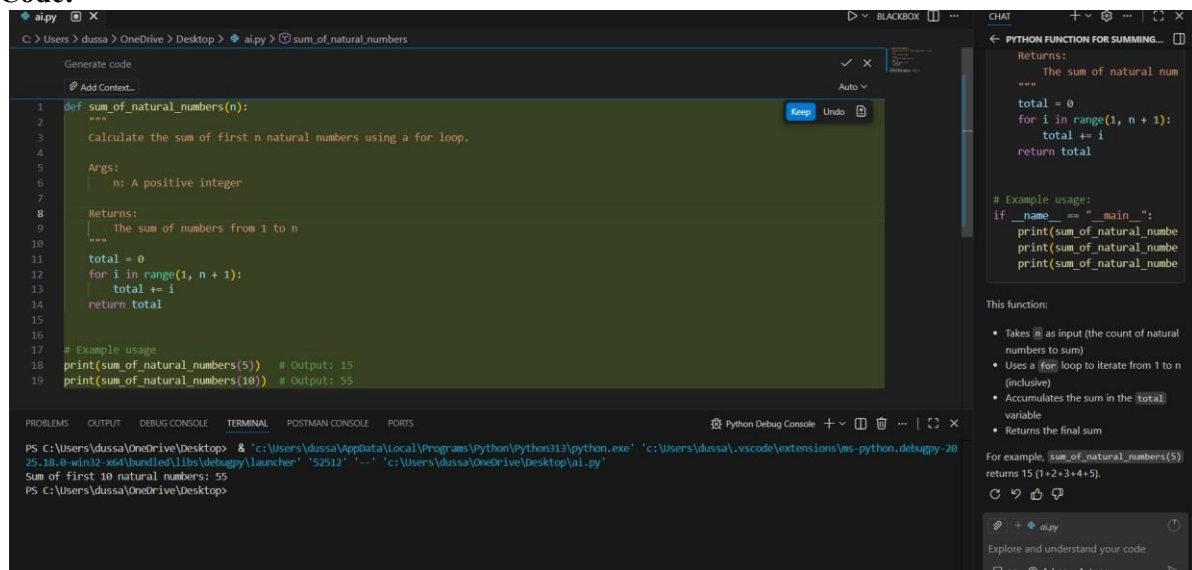
Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

Prompt:

Write a Python function to calculate the sum of first n natural numbers using a for loop.

Code:



```
def sum_of_natural_numbers(n):  
    """  
    calculate the sum of first n natural numbers using a for loop.  
    """  
    Args:  
        n: A positive integer  
    Returns:  
        The sum of numbers from 1 to n  
    """  
    total = 0  
    for i in range(1, n + 1):  
        total += i  
    return total  
  
# Example usage  
print(sum_of_natural_numbers(5)) # Output: 15  
print(sum_of_natural_numbers(10)) # Output: 55
```

Python Debug Console

```
PS C:\Users\dussal\OneDrive\Desktop> & 'c:\Users\dussal\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussal\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\lib\debugpy\launcher' '52512' '--' 'c:\Users\dussal\OneDrive\Desktop\al.py'  
Sum of first 10 natural numbers: 55  
PS C:\Users\dussal\OneDrive\Desktop>
```

CHAT

PYTHON FUNCTION FOR SUMMING...

Returns:
The sum of natural num
total = 0
for i in range(1, n + 1):
 total += i
return total

Example usage:
if __name__ == "__main__":
 print(sum_of_natural_numbe
 print(sum_of_natural_numbe
 print(sum_of_natural_numbe

This function:

- Takes `n` as input (the count of natural numbers to sum)
- Uses a `for` loop to iterate from 1 to `n` (inclusive)
- Accumulates the sum in the `total` variable
- Returns the final sum

For example, `sum_of_natural_numbers(5)` returns 15 (1+2+3+4+5).

Using Mathematical Formula

```
def sum_to_n_formula(n):  
    return n * (n + 1) // 2
```

Analysis

The formula-based approach is the most efficient with $O(1)$ time complexity. Loop-based methods are easier to understand for beginners.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as `deposit()`, `withdraw()`, and `check_balance()`.
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.

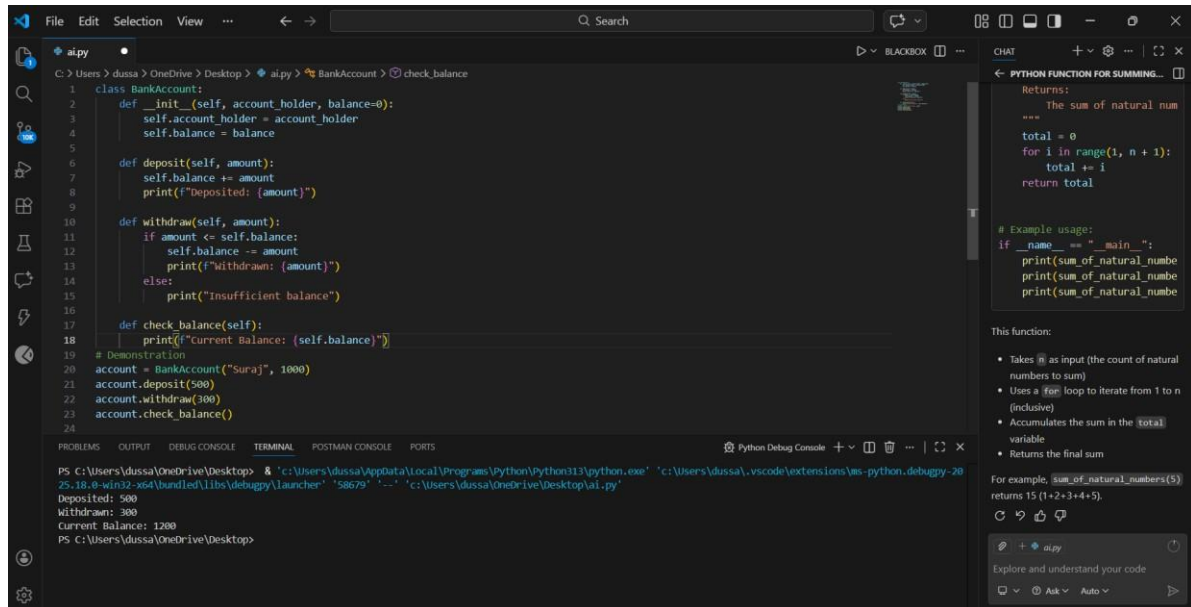
- Well-commented code with a clear explanation.

Prompt:

Generate a Python BankAccount class with deposit, withdraw, and check_balance methods.

Code:

AI-Generated Code with Comments



```
File Edit Selection View ... Search
C:\Users\dussa> OneDrive\ Desktop > ai.py > BankAccount > check_balance

1 class BankAccount:
2     def __init__(self, account_holder, balance=0):
3         self.account_holder = account_holder
4         self.balance = balance
5
6     def deposit(self, amount):
7         self.balance += amount
8         print(f"Deposited: {amount}")
9
10    def withdraw(self, amount):
11        if amount <= self.balance:
12            self.balance -= amount
13            print(f"Withdrawn: {amount}")
14        else:
15            print("Insufficient balance")
16
17    def check_balance(self):
18        print(f"Current Balance: {self.balance}")
19
20    # Demonstration
21    account = BankAccount("Suraj", 1000)
22    account.deposit(500)
23    account.withdraw(300)
24    account.check_balance()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS

Python Debug Console

```
PS C:\Users\dussa\OneDrive\Desktop> 'c:\Users\dussa\AppData\Local\Programs\Python\Python113\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.8-win32-x64\bundled\libs\debugpy\launcher' '58679' '-' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
Deposited: 500
Withdrawn: 300
Current Balance: 1200
PS C:\Users\dussa\OneDrive\Desktop>
```

CHAT

PYTHON FUNCTION FOR SUMMING...

Returns:
The sum of natural num

```
"""
total = 0
for i in range(1, n + 1):
    total += i
return total

# Example usage:
if __name__ == "__main__":
    print(sum_of_natural_numbe
    print(sum_of_natural_numbe
    print(sum_of_natural_numbe
```

This function:

- Takes `n` as input (the count of natural numbers to sum)
- Uses a `for` loop to iterate from 1 to `n` (inclusive)
- Accumulates the sum in the `total` variable
- Returns the final sum

For example, `sum_of_natural_numbers(5)` returns 15 (1+2+3+4+5).

Explore and understand your code

Ask Auto

Explanation

The class correctly encapsulates account data and operations. Conditional checks prevent invalid withdrawals. The code is clean, readable, and suitable for a basic banking application.